
🔴 **SAVOL 11:**

`PYQT5` DA `TCP SOCKET` USTIDAN GUI BOSHQARISH: `QTHREAD` YOKI `SIGNAL-SLOT` YORDAMIDA TARMOQ HODISALARINI GUI GA YETKAZISH. NIMA UCHUN SOCKET OPERATSIYALARINI ASOSIY `THREAD` DA BAJARISH XAVFLI?

---

1. Muammo — Nima uchun Asosiy Threadda Socket Xavfli?

PyQt5 da **asosiy thread** — bu Qt ning **Event Loop** i ishlaydigan thread. U faqat bitta vazifa bajaradi: foydalanuvchi harakatlarini (klik, klaviatura, chizish) qayta ishlash.

```
Asosiy Thread (Qt Event Loop):
    ↓
Tugma bosildi → hodisani qayta ishlash → ekranni yangilash
Sichqoncha → hodisani qayta ishlash → ekranni yangilash
...chizish, yangilash, javob berish...
```

Agar shu threadda `recv()` chaqirsak:

```
Asosiy Thread:
    ↓
recv() ← BU YERDA TO'XTAYDI... kutadi... kutadi...
    ↓
Qt Event Loop ishlamayapti!
    ↓
Oyna muzlaydi — foydalanuvchi hech narsa qila olmaydi
Tugmalar ishlamaydi, oyna tortib bo'lmaydi
Windows: "Dastur javob bermayapti" xabari
```

Nazariy asos: `recv()` — **blokirovchi** funksiya. U ma'lumot kelgunga qadar thread ni to'xtatib qo'yadi. Qt Event Loop esa to'xtasa — butun GUI muzlaydi. Bu ikki narsa bir threadda yashay olmaydi.

```
Noto'g'ri:                    To'g'ri:

Main Thread:                  Main Thread:       Worker Thread:
  Qt Event Loop    →            Qt Event Loop  +   recv() kutadi
  recv() (muzlaydi)→            GUI ishlaydi       ma'lumot keldi
  GUI muzlaydi     →            Signal yuborildi ← signal emit()
                                GUI yangilanadi
```

---

2. Yechim — QThread

QThread — PyQt5 ning o'z thread klassi. U Python ning `threading.Thread` idan farqli — Qt Signal-Slot mexanizmi bilan to'liq integratsiyalashgan.

```
QThread ning afzalligi:
  → Signal-Slot orqali GUI bilan xavfsiz gaplashadi
```

```
    → Qt tomonidan boshqariladi
    → Thread-safe signal yuborish o'rnatilgan
```

Eng muhim qoida:

```
Worker Thread  →  faqat hisoblash, tarmoq, fayl
Main Thread    →  faqat GUI (widget, label, button)

Worker Thread dan to'g'ridan-to'g'ri widget ga teginish → XAVFLI!
Worker Thread → Signal emit() → Main Thread → widget yangilash → XAVFSIZ!
```

---

3. QThread dan To'g'ri Foydalanish

QThread dan foydalanishning **2 usuli** bor:

❌ Noto'g'ri usul — QThread ni subclass qilish:

```python
class MyThread(QThread):
    def run(self):
        # Hamma ish shu yerda
        while True:
            data = sock.recv(1024)  # blokirovchi
```

Bu usul ishlaydi, lekin Qt arxitekturasi nuqtai nazaridan **noto'g'ri** — `run()` ni override qilish QThread ning ichki mexanizmlarini buzishi mumkin.

✅ To'g'ri usul — Worker ob'ektini threadga ko'chirish:

```python
from PyQt5.QtCore import QObject, QThread, pyqtSignal

# 1. Worker — faqat ish bajaradi
class SocketWorker(QObject):
    # Signallar — Main Thread ga xabar yuborish uchun
    xabar_keldi   = pyqtSignal(str)    # yangi xabar
    ulandi        = pyqtSignal()        # ulanish o'rnatildi
    uzildi        = pyqtSignal(str)     # ulanish uzildi + sabab
    xato_yuz_berdi= pyqtSignal(str)    # xato xabari

    def __init__(self, host, port):
        super().__init__()
        self.host   = host
        self.port   = port
        self.sock   = None
        self._ishla = True   # to'xtatish uchun flag

    def ishlat(self):   # ← Thread ichida chaqiriladi
        """Asosiy ish — tarmoq operatsiyalari"""
        try:
            self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.sock.settimeout(5.0)
            self.sock.connect((self.host, self.port))
            self.sock.settimeout(1.0)   # recv uchun kichik timeout
            self.ulandi.emit()          # ← GUI ga: "Ulandik!"
```

```
            while self._ishla:
                try:
                    data = self.sock.recv(1024)
                    if not data:
                        self.uzildi.emit("Server ulanishni yopdi")
                        break
                    self.xabar_keldi.emit(data.decode("utf-8"))

                except socket.timeout:
                    continue   # timeout — davom et, to'xtatish tekshiriladi

        except ConnectionRefusedError:
            self.xato_yuz_berdi.emit("Server topilmadi")
        except OSError as e:
            self.xato_yuz_berdi.emit(f"Tarmoq xatosi: {e}")
        finally:
            if self.sock:
                self.sock.close()

    def xabar_yuborish(self, matn: str):
        """Main Thread dan chaqiriladi — xavfsizmi?"""
        # send() odatda tez, blokirovchi emas
        # lekin to'g'risi: bu ham worker threadda bajarilishi kerak
        if self.sock:
            try:
                self.sock.sendall(matn.encode("utf-8"))
            except OSError:
                self.uzildi.emit("Yuborishda xato")

    def toqtat(self):
        self._ishla = False


# 2. Main Window — faqat GUI
class ChatOyna(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TCP Chat")
        self.setGeometry(100, 100, 500, 600)
        self.worker = None
        self.thread = None
        self._interfeys_yaratish()

    def _interfeys_yaratish(self):
        markaziy = QWidget()
        self.setCentralWidget(markaziy)
        tartib = QVBoxLayout(markaziy)

        # Chat maydoni
        self.chat_maydoni = QTextEdit()
        self.chat_maydoni.setReadOnly(True)
        tartib.addWidget(self.chat_maydoni)

        # Xabar yozish
        yozish_qatori = QHBoxLayout()
        self.xabar_input = QLineEdit()
        self.xabar_input.setPlaceholderText("Xabar yozing...")
```

```
        self.yuborish_btn = QPushButton("Yuborish")
        self.yuborish_btn.setEnabled(False)
        yozish_qatori.addWidget(self.xabar_input)
        yozish_qatori.addWidget(self.yuborish_btn)
        tartib.addLayout(yozish_qatori)

        # Ulanish tugmasi
        self.ulanish_btn = QPushButton("Ulanish")
        tartib.addWidget(self.ulanish_btn)

        # Signal-Slot
        self.ulanish_btn.clicked.connect(self.ulanishni_boshlash)
        self.yuborish_btn.clicked.connect(self.xabar_yuborish)
        self.xabar_input.returnPressed.connect(self.xabar_yuborish)

    def ulanishni_boshlash(self):
        # 1. Thread va Worker yaratish
        self.thread = QThread()
        self.worker = SocketWorker("127.0.0.1", 12345)

        # 2. Worker ni threadga ko'chirish ← muhim!
        self.worker.moveToThread(self.thread)

        # 3. Signal-Slot bog'lash
        # Thread signallari
        self.thread.started.connect(self.worker.ishlat)

        # Worker → GUI signallari (thread-safe avtomatik!)
        self.worker.ulandi.connect(self.ulanish_holati)
        self.worker.xabar_keldi.connect(self.xabar_qabul_qilish)
        self.worker.uzildi.connect(self.uzilish_holati)
        self.worker.xato_yuz_berdi.connect(self.xato_korsatish)

        # Tozalash
        self.worker.uzildi.connect(self.thread.quit)
        self.thread.finished.connect(self.thread.deleteLater)

        # 4. Threadni ishga tushirish
        self.thread.start()
        self.ulanish_btn.setEnabled(False)
        self.log("🔄 Ulanish...")

    # ── GUI Slotlari (Main Thread da ishlaydi) ──

    def ulanish_holati(self):
        self.yuborish_btn.setEnabled(True)
        self.log("✅ Server bilan ulandi!")

    def xabar_qabul_qilish(self, matn: str):
        self.log(f"📨 {matn}")

    def uzilish_holati(self, sabab: str):
        self.yuborish_btn.setEnabled(False)
        self.ulanish_btn.setEnabled(True)
        self.log(f"🔌 Uzildi: {sabab}")

    def xato_korsatish(self, xabar: str):
```

```
        self.log(f"❌ Xato: {xabar}")
        self.ulanish_btn.setEnabled(True)

    def xabar_yuborish(self):
        matn = self.xabar_input.text().strip()
        if matn and self.worker:
            self.worker.xabar_yuborish(matn)
            self.log(f"🗣️ Men: {matn}")
            self.xabar_input.clear()

    def log(self, matn: str):
        self.chat_maydoni.append(matn)

    def closeEvent(self, event):
        if self.worker:
            self.worker.toqtat()
        if self.thread:
            self.thread.quit()
            self.thread.wait()
        event.accept()
```

---

4. moveToThread() — Eng Muhim Tushuncha

`self.worker.moveToThread(self.thread)`

Bu qator nimani anglatadi?

```
Oldin:
  worker ob'ekti → Main Thread da yashaydi

moveToThread() dan keyin:
  worker ob'ekti → Worker Thread da yashaydi

Natija:
  worker.ishlat() → Worker Thread da bajariladi ✅
  worker.signal.emit() → Qt avtomatik Main Thread ga o'tkazadi ✅
  self.label.setText() → Main Thread da bajariladi ✅
```

Signal-Slot da Thread Xavfsizligi:

```
Worker Thread:          Main Thread:
  signal.emit(data)  →  [Qt ichki navbat]  →  slot(data) chaqiriladi

Qt bu o'tkazishni avtomatik bajaradi!
Biz qo'lda lock, mutex ishlatishimiz shart emas.
Bu PyQt5 ning eng kuchli tomoni.
```

---

5. Xato Holat — Asosiy Threaddan Widget Yangilash

```
# ❌ XAVFLI — Worker Thread dan to'g'ridan-to'g'ri widget:
class SocketWorker(QObject):
    def ishlat(self):
        data = self.sock.recv(1024)
```

```
        self.label.setText(data.decode())  # ← XAVFLI!
        # Worker Thread → GUI widget → CRASH yoki kutilmagan xatti-harakat

# ✅ XAVFSIZ — Signal orqali:
class SocketWorker(QObject):
    xabar_keldi = pyqtSignal(str)

    def ishlat(self):
        data = self.sock.recv(1024)
        self.xabar_keldi.emit(data.decode())  # ← Worker Thread dan signal
        # Qt avtomatik Main Thread ga yetkazadi → label.setText() xavfsiz
```

---

6. Xulosa

```
Nima uchun asosiy threadda socket xavfli?
  → recv() blokirovchi — thread to'xtaydi
  → Qt Event Loop to'xtaydi
  → GUI muzlaydi, oyna javob bermaydi

Yechim — QThread + moveToThread():
  → Socket operatsiyalari Worker Thread da
  → GUI operatsiyalari Main Thread da
  → Signal-Slot ikkalasini xavfsiz bog'laydi

Arxitektura:
  Worker Thread              Main Thread
  ──────────────             ────────────
  sock.connect()    signal→  ulanish_holati() → btn.setEnabled()
  sock.recv()       signal→  xabar_keldi()    → chat.append()
  xato yuz berdi    signal→  xato_korsatish()→ QMessageBox
```

> 💡 **Eslab qol:** PyQt5 da oltin qoida — **faqat Main Thread widget bilan ishlaydi, faqat Worker Thread tarmoq bilan ishlaydi.** Ular orasidagi ko'prik — Signal-Slot!

🔴 **SAVOL 12:**

PYTHON ILOVASINI MUSTAQIL YUKLANUVCHI FAYLGA AYLANTIRISH: `PYINSTALLER` VA `CX_FREEZE` QIYOSIY TAHLILI. `SPEC` FAYL, `--ONEFILE` REJIMI, RESURSLARNI TO'G'RI PAKETLASH MUAMMOLARI.

---

1. Nima uchun Paketlash Kerak?

Python dasturini boshqa kompyuterda ishlatish uchun odatda Python o'rnatilgan bo'lishi kerak. Paketlash — bu dasturni Python o'rnatilmagan kompyuterlarda ham ishlaydigan **mustaqil faylga** aylantirish jarayoni.

```
Oddiy holat:
  main.py → ishlashi uchun → Python + PyQt5 + barcha kutubxonalar kerak
```

```
Paketlangandan keyin:
  main.exe → ishlashi uchun → hech narsa kerak emas!
  (Python interpreter + kutubxonalar ichiga joylashtirilgan)
```

---

2. Paketlash Qanday Ishlaydi — Nazariy Asos

Paketlovchi dasturlar quyidagi ishlarni bajaradi:

```
1. Tahlil bosqichi:
   main.py → barcha import lar aniqlanadi
   (PyQt5, socket, os, sys va boshqalar)

2. Yig'ish bosqichi:
   → Python interpreter (python.dll / libpython.so)
   → Barcha kerakli .pyc fayllar
   → Barcha kerakli .dll / .so kutubxonalar
   → Resurslar (rasm, ikon, ma'lumotlar)

3. Paketlash bosqichi:
   → Hammasi bitta papka yoki bitta faylga joylashtiriladi

4. Natija:
   → Windows: main.exe
   → Linux:   main (executable)
   → macOS:   main.app
```

---

3. PyInstaller — To'liq Tavsif

PyInstaller — eng keng tarqalgan Python paketlovchisi. PyQt5 bilan juda yaxshi ishlaydi.

O'rnatish:

```
pip install pyinstaller
```

Asosiy ishlatish:

```
# Oddiy paketlash (papka yaratadi)
pyinstaller main.py

# Bitta fayl rejimi
pyinstaller --onefile main.py

# Konsolsiz (GUI dasturlar uchun)
pyinstaller --onefile --windowed main.py

# Ikon bilan
pyinstaller --onefile --windowed --icon=logo.ico main.py

# Barchasi birgalikda
pyinstaller --onefile --windowed --icon=logo.ico --name="ChatDastur" main.py
```

Paketlash natijalari:

```
loyiha/
├── main.py
├── main.spec          ← avtomatik yaratiladi
├── build/             ← vaqtinchalik fayllar
│   └── main/
└── dist/              ← tayyor dastur shu yerda!
    └── main.exe       (--onefile da)
    yoki
    └── main/          (oddiy rejimda)
        ├── main.exe
        ├── PyQt5/
        └── ...
```

---

4. --onefile va --onedir Rejimlari

--onedir (standart rejim):

```
dist/
└── main/
    ├── main.exe         ← asosiy fayl
    ├── python38.dll
    ├── PyQt5/
    │   ├── Qt5Core.dll
    │   ├── Qt5Widgets.dll
    │   └── ...
    └── (yuzlab boshqa fayllar)
```

Qanday ishlaydi: Dastur ishga tushganda hamma fayl allaqachon papkada tayyor. Tez ishga tushadi.

Afzalligi: Tez ishga tushadi, debug qilish oson. **Kamchiligi:** Ko'p fayllar — foydalanuvchiga berish noqulay.

--onefile rejimi:

```
dist/
└── main.exe    ← faqat BITTA fayl, ichida hamma narsa!
```

Qanday ishlaydi — muhim nazariy tushuncha:

```
Foydalanuvchi main.exe ni ishga tushiradi
        ↓
PyInstaller bootloader ishga tushadi
        ↓
Vaqtinchalik papka yaratiladi:
  Windows: C:\Users\User\AppData\Local\Temp\_MEIxxxxxx\
  Linux:   /tmp/_MEIxxxxxx/
        ↓
Barcha fayllar shu papkaga chiqariladi (extract)
        ↓
```

```
Dastur ishga tushadi
        ↓
Dastur yopilganda vaqtinchalik papka o'chiriladi
```

Afzalligi: Bitta fayl — berish, tarqatish oson. **Kamchiligi:** Har ishga tushganda extraction — sekinroq (~2-5 soniya).

---

5. SPEC Fayl — Kuchli Boshqaruv Vositasi

spec fayl — PyInstaller ning konfiguratsiya fayli. Avval `pyinstaller main.py` ishga tushirilganda avtomatik yaratiladi. Keyingi marta esa spec fayl orqali boshqarish mumkin.

```
# main.spec fayli

# Analysis — nima paketlanishini aniqlaydi
a = Analysis(
    ['main.py'],                    # asosiy fayl
    pathex=['/loyiha/papkasi'],     # qo'shimcha qidiruv yo'llari
    binaries=[],                    # qo'shimcha .dll/.so fayllar
    datas=[                         # resurslar (rasm, ma'lumot va h.k.)
        ('images/', 'images/'),     # (manba, maqsad) juft
        ('config.json', '.'),       # config ni ildiz papkaga
        ('icons/*.ico', 'icons/'),  # barcha ikonkalar
    ],
    hiddenimports=[                 # PyInstaller topa olmagan importlar
        'PyQt5.sip',
        'pkg_resources.py2_warn',
    ],
    hookspath=[],
    excludes=[                      # keraksiz modullarni chiqarish
        'tkinter',
        'matplotlib',
        'numpy',
    ],
)

# PYZ — Python fayllarni arxivlaydi
pyz = PYZ(a.pure, a.zipped_data)

# EXE — bajariladigan fayl
exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.zipfiles,
    a.datas,
    name='ChatDastur',
    debug=False,
    strip=False,
    upx=True,               # UPX siqish (fayl kichiklashadi)
    console=False,          # True = konsol ko'rinadi
    icon='logo.ico',
    onefile=True,
)
```

Spec fayl orqali ishlatish:

```
pyinstaller main.spec
```

---

6. Resurslarni To'g'ri Paketlash — Eng Muhim Muammo

Bu **eng ko'p uchraydigan muammo**. `--onefile` rejimida dastur vaqtinchalik papkaga chiqariladi, shuning uchun oddiy yo'llar ishlamaydi.

❌ Noto'g'ri usul:

```
# Bu faqat development da ishlaydi!
rasm = QPixmap("images/logo.png")
```

Paketlangandan keyin `images/logo.png` yo'li topilmaydi — chunki fayl vaqtinchalik papkada boshqa joyda.

✅ To'g'ri usul — resource_path funksiyasi:

```
import sys
import os

def resource_path(nisbiy_yol: str) -> str:
    """
    Har ikkala holatda ham to'g'ri yo'lni qaytaradi:
    1. Development (python main.py)
    2. Paketlangan (main.exe --onefile)
    """
    if hasattr(sys, '_MEIPASS'):
        # PyInstaller vaqtinchalik papkasi
        # --onefile da: C:\Temp\_MEIxxxxxx\
        asos = sys._MEIPASS
    else:
        # Oddiy Python ishga tushirish
        asos = os.path.abspath(".")

    return os.path.join(asos, nisbiy_yol)

# Ishlatish:
rasm      = QPixmap(resource_path("images/logo.png"))
ikon      = QIcon(resource_path("icons/app.ico"))
config    = open(resource_path("config.json"))
```

Spec faylda resurslarni ko'rsatish:

```
datas=[
    ('images/', 'images/'),      # images papkasini to'liq paketlash
    ('icons/', 'icons/'),        # icons papkasini to'liq paketlash
    ('config.json', '.'),        # config.json ni ildizga
    ('data/*.db', 'data/'),      # barcha .db fayllar
],
```

---

7. HiddenImports — Ko'rinmas Importlar Muammosi

Ba'zi modullar **dinamik** import qilinadi — PyInstaller ularni topa olmaydi:

```
# Dinamik import — PyInstaller ko'rmaydi:
modul_nomi = "PyQt5.QtNetwork"
modul = __import__(modul_nomi)        # ← PyInstaller buni tushunmaydi

# Yoki plugin tizimlar:
importlib.import_module("plugin_" + ism)
```

Yechim — spec faylda yoki buyruqda ko'rsatish:

```
# spec faylda:
hiddenimports=[
    'PyQt5.sip',
    'PyQt5.QtNetwork',
    'PyQt5.QtMultimedia',
    'socket',
    'json',
    'pkg_resources.py2_warn',
]
# Buyruq qatorida:
pyinstaller --onefile --hidden-import PyQt5.sip main.py
```

---

8. cx_Freeze — Qiyosiy Tahlil

cx_Freeze — PyInstaller ga alternativ. Setup skript yozish talab qiladi.

O'rnatish:

```
pip install cx_Freeze
```

setup.py fayli:

```
from cx_Freeze import setup, Executable
import sys

# Windows uchun base
base = "Win32GUI" if sys.platform == "win32" else None

# Qo'shimcha fayllar
include_files = [
    ("images/", "images/"),
    ("config.json", "config.json"),
]

# Paket sozlamalari
build_options = {
    "packages": ["PyQt5", "socket"],
    "excludes": ["tkinter", "unittest"],
    "include_files": include_files,
}

setup(
```

```
    name="ChatDastur",
    version="1.0",
    description="TCP Chat Dasturi",
    options={"build_exe": build_options},
    executables=[
        Executable(
            "main.py",
            base=base,
            icon="logo.ico",
            target_name="ChatDastur.exe",
        )
    ]
)
# Ishlatish:
python setup.py build
# Natija: build/exe.win-amd64-3.x/ papkasida
```

---

9. PyInstaller vs cx_Freeze — To'liq Taqqoslash

Xususiyat	PyInstaller	cx_Freeze
O'rnatish qulayligi	✅ Oson	✅ Oson
Bitta fayl rejimi	✅ `--onefile`	❌ Yo'q
Konfiguratsiya	✅ spec fayl	✅ setup.py
PyQt5 qo'llab-quvvatlash	✅ Kuchli	✅ Yaxshi
Platforma	✅ Win/Mac/Linux	✅ Win/Mac/Linux
Fayl hajmi	⚠️ Katta (~50MB)	⚠️ Katta (~50MB)
Ishga tushish tezligi	⚠️ Sekinroq (onefile)	✅ Tezroq
Jamoa hajmi	✅ Katta	⚠️ Kichikroq
Hujjatlar	✅ Ko'p	⚠️ Kam
Kichik loyiha	✅ Ideal	✅ Yaxshi
`--onefile` kerak bo'lsa	✅	❌

---

10. Keng Tarqalgan Muammolar va Yechimlari

Muammo 1 — Antivirus False Positive:

```
Muammo: Antivirus dastur main.exe ni virus deb belgilaydi
Sabab:  PyInstaller bootloader ko'p viruslarda ham ishlatiladi

Yechim:
  → PyInstaller ni manba koddan o'zi kompilyatsiya qilish
  → Code signing sertifikati olish (pullik)
  → Foydalanuvchiga antivirusga exception qo'shishni aytish
```

Muammo 2 — Fayl hajmi katta:

```
Muammo: main.exe hajmi 80-150MB
Sabab:  PyQt5 o'zi ~50MB, Python ~30MB

Yechim:
  → UPX siqish: pyinstaller --upx-dir=/upx main.py (~30% kichiklashadi)
  → Keraksiz modullarni chiqarish (excludes)
  → Virtual environment da faqat kerakli paketlar o'rnatish
```

Muammo 3 — DLL topilmadi xatosi:

```
Muammo: "VCRUNTIME140.dll not found" xatosi
Sabab:  Visual C++ Runtime o'rnatilmagan

Yechim:
  → Microsoft Visual C++ Redistributable o'rnatish
  → Yoki installer (NSIS, Inno Setup) bilan birga tarqatish
```

Muammo 4 — Resurs fayl topilmadi:

```python
# Muammo: FileNotFoundError: images/logo.png
# Sabab: --onefile da yo'l o'zgaradi

# Yechim: resource_path() funksiyasini ishlatish (yuqorida keltirilgan)
rasm = QPixmap(resource_path("images/logo.png"))
```

Muammo 5 — Konsol oynasi ko'rinib qoladi:

```
# Muammo: GUI dasturda qora konsol oynasi ham ochiladi
# Yechim:
pyinstaller --windowed main.py
# yoki spec faylda:
exe = EXE(..., console=False, ...)
```

---

11. Amaliy Tavsiyalar — Qadamba-Qadam

```
# 1. Avval oddiy test:
pyinstaller main.py
# dist/main/ papkasini tekshiring — ishlayaptimi?

# 2. Muvaffaqiyatli bo'lsa — onefile:
pyinstaller --onefile --windowed --icon=logo.ico main.py

# 3. Resurslar kerak bo'lsa — spec fayl tahriri:
# main.spec da datas=[...] ni to'ldiring
pyinstaller main.spec

# 4. Boshqa kompyuterda test qiling!
# (Python o'rnatilmagan kompyuterda)
```

---

12. Xulosa

```
PyInstaller:
  ✅ --onefile rejimi — bitta .exe fayl
  ✅ spec fayl — kuchli boshqaruv
  ✅ PyQt5 bilan zo'r ishlaydi
  ✅ Keng jamoa, ko'p hujjatlar
  ⚠️ Sekinroq ishga tushish (onefile da)

cx_Freeze:
  ✅ setup.py — tanish Python uslubi
  ✅ Tezroq ishga tushish
  ❌ --onefile rejimi yo'q
  ⚠️ Kamroq hujjatlar

Eng muhim qoidalar:
  1. resource_path() → har doim resurs yo'llari uchun
  2. sys._MEIPASS   → onefile da vaqtinchalik papka
  3. hiddenimports  → dinamik importlar uchun
  4. --windowed     → GUI dasturlarda konsol yashirish
  5. Boshqa kompyuterda sinab ko'rish → shart!
```

> 💡 **Eslab qol:** Paketlash — bu "bir marta qilib, hamma joyda ishlaydi" emas. Har doim **Python o'rnatilmagan toza kompyuterda** sinab ko'rish shart. Aks holda kutilmagan xatolar foydalanuvchida paydo bo'ladi!

🔴 **SAVOL 13:**

`TCP` VA `UDP` PROTOKOLLARINING FARQLARI, QAYSI HOLATLARDA QAYSI BIRINI ISHLATISH KERAK? CHAT ILOVASI UCHUN `TCP` NI TANLAGANINGIZNING ASOSIY SABABLARI NIMA?

---

1. Ikki Protokol — Ikki Falsafa

TCP va UDP — ikkalasi ham **Transport qatlami** protokollari. Lekin ularning ishlash falsafasi tubdan farq qiladi:

```
TCP → "Xat jo'natish" — yetib borishi kafolatlanadi,
       yo'qolsa qayta yuboriladi, tartib saqlanadi.
       Lekin vaqt oladi.

UDP → "Radio eshittirish" — yuboriladi va unutiladi.
       Yetib bordi-yetmadi — tekshirilmaydi.
       Lekin juda tez.
```

---

2. TCP — Qanday Ishlaydi?

Ulanish o'rnatish — 3 bosqichli Handshake:

```
Klient          Server
  │──── SYN ────→│     "Ulanmoqchiman"
```

```
  │←── SYN+ACK ──│      "Mayli"
  │──── ACK ────→│      "Boshlaylik"
  │              │
  │═══ Ma'lumot ═│      Xavfsiz kanal ochiq
```

Ma'lumot yetkazilishini kafolatlash:

```
Klient                    Server
  │──── Paket 1 ──────────→│
  │←─── ACK 1 ─────────────│   "1-chi yetdi"
  │──── Paket 2 ──────────→│
  │                        │   (ACK kelmadi — yo'qoldi!)
  │──── Paket 2 ──────────→│   Qayta yuboriladi
  │←─── ACK 2 ─────────────│   "2-chi yetdi"
```

Tartib kafolati:

```
Yuborildi:   [1] [3] [2] [4]   (tarmoqda aralashib ketdi)
Qabul qilindi: [1] [2] [3] [4] (TCP tartibga keltiradi)
```

TCP ning asosiy xususiyatlari:

- ✅ Ulanish o'rnatiladi (connection-oriented)
- ✅ Yetkazilish kafolatlanadi (reliable)
- ✅ Tartib saqlanadi (ordered)
- ✅ Xatolar tekshiriladi (error checking)
- ✅ Oqim boshqaruvi (flow control) — tez yuboruvchi sekin qabul qiluvchini bosib ketmaydi
- ✅ Tiqilib qolish boshqaruvi (congestion control)
- ❌ Nisbatan sekin
- ❌ Qo'shimcha sarflar (overhead) katta

---

3. UDP — Qanday Ishlaydi?

UDP da hech qanday ulanish o'rnatilmaydi. Paket yuboriladi va unutiladi:

```
Klient                    Server
  │──── Paket 1 ──────────→│   (yetdi yoki yetmadi — bilmaymiz)
  │──── Paket 2 ─────X     │   (yo'qoldi — hech kim bilmaydi)
  │──── Paket 3 ──────────→│
  │──── Paket 4 ──────────→│

Natija: Server 1, 3, 4 paketlarni oldi.
        2-chi yo'qoldi — hech kim xabar bermaydi.
```

UDP ning asosiy xususiyatlari:

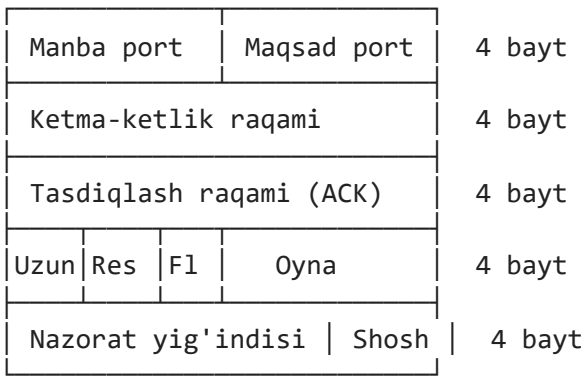
- ❌ Ulanish o'rnatilmaydi (connectionless)
- ❌ Yetkazilish kafolatlanmaydi (unreliable)
- ❌ Tartib saqlanmaydi
- ❌ Xatolar tekshirilmaydi (minimal)
- ✅ Juda tez

✅ Overhead minimal (sarlavha 8 bayt, TCP da 20 bayt)
✅ Broadcast va Multicast qo'llab-quvvatlaydi
(bir paket → ko'p qabul qiluvchi)

---

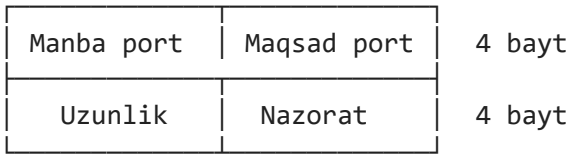
4. Sarlavha Hajmi — Texnik Farq

TCP sarlavhasi (20 bayt minimum):



Jami: 20 bayt (+ ixtiyoriy)

UDP sarlavhasi (8 bayt):



Jami: 8 bayt

Natija: UDP 2.5x kichikroq sarlavha → tezroq

---

5. Qaysi Holatda Qaysi Protokol?

TCP ishlatish kerak bo'lgan holatlar:

Chat ilovasi:
→ Xabar yo'qolsa bo'lmaydi
→ Tartib muhim ("Salom" → "Xayr" tartibida kelishi kerak)
→ TCP ✅

Veb sayt (HTTP/HTTPS):
→ HTML, CSS, JS to'liq kelishi shart
→ Bitta bayt yo'qolsa — sahifa buziladi
→ TCP ✅

Elektron pochta (SMTP, IMAP):
→ Xat to'liq yetib borishi shart
→ TCP ✅

Fayl uzatish (FTP, SCP):
→ Fayl 1 bayt ham yo'qolsa buziladi
→ TCP ✅

```
Ma'lumotlar bazasi so'rovlari:
  → So'rov va javob to'liq bo'lishi shart
  → TCP ✅
```

UDP ishlatish kerak bo'lgan holatlar:

```
Online o'yinlar (real vaqt):
  → Oyinchi pozitsiyasi har 16ms yangilanadi
  → 1 paket yo'qolsa — keyingisi keladi (eskirgan ma'lumot)
  → Kafolat emas, tezlik muhim
  → UDP ✅

Video qo'ng'iroq (Zoom, Skype):
  → 1-2 ta kadr yo'qolsa — insonlar sezmaydi
  → Kechikish (lag) esa juda seziladi
  → UDP ✅

Audio streaming (radio, musiqa):
  → Bir lahzalik statik — muammo emas
  → Buferlash bilan qoplanadi
  → UDP ✅

DNS so'rovlari:
  → Kichik so'rov, kichik javob
  → Ulanish o'rnatish ortiqcha xarajat
  → UDP ✅

IoT sensorlar:
  → Har soniyada temperatura yuboriladi
  → 1 ta yo'qolsa — keyingisi keladi
  → UDP ✅
```

---

6. Aralash Yondashuv — Real Dunyo

Ba'zi ilovalar **ikkalasini birga** ishlatadi:

```
Online o'yin (masalan PUBG, Fortnite):
  → Harakat, pozitsiya, o'q → UDP (tez, yo'qolsa ham bo'ladi)
  → Login, o'yin natijasi, to'lov → TCP (kafolat kerak)

Video qo'ng'iroq (Zoom):
  → Video/audio oqimi → UDP (tezlik muhim)
  → Chat xabarlari → TCP (yo'qolmasligi kerak)

QUIC protokoli (HTTP/3):
  → UDP ustiga TCP imkoniyatlari qurilgan
  → Google tomonidan ishlab chiqilgan
  → Zamonaviy kompromiss yechim
```

---

7. Chat Ilovasi Uchun TCP — Asosiy Sabablar

Sabab 1 — Xabar yo'qolmasligi shart:

```
Tasavvur qiling:
  Ali: "Ertaga soat 5 da uchrashaylik"
  Bobur: (xabar yo'qoldi — UDP)
  Bobur: "Kelmasang-chi?" (keyingi xabar)

  → Bobur nima soatda kelishini bilmaydi!
  → Chat uchun bu qabul qilinmas

TCP bilan:
  → Har xabar yetib borishi kafolatlanadi
  → Yo'qolsa — avtomatik qayta yuboriladi
  → Bobur har doim to'liq kontekstni ko'radi ✅
```

Sabab 2 — Tartib muhim:

```
UDP da tartib buzilishi:
  Yuborildi:     "Salom" → "Qalaysan?" → "Yaxshi"
  Qabul qilindi: "Yaxshi" → "Salom" → "Qalaysan?"

  → Suhbat mantiqsiz bo'lib qoladi!

TCP da tartib kafolatlanadi:
  Yuborildi:     "Salom" → "Qalaysan?" → "Yaxshi"
  Qabul qilindi: "Salom" → "Qalaysan?" → "Yaxshi" ✅
```

Sabab 3 — Ulanish holati muhim:

```
Chat da kim onlayn/offlayn ekanini bilish kerak.
TCP ulanishi uzilganda darhol bilinadi:
  → Foydalanuvchi chiqib ketdi → TCP connection closed
  → Server "XYZ offlayn bo'ldi" deb xabar beradi

UDP da:
  → Ulanish tushunchasi yo'q
  → Kim onlayn, kim offlayn — aniqlab bo'lmaydi
  → Qo'lda "heartbeat" mexanizmi kerak (murakkab)
```

Sabab 4 — Kechikish chat uchun muhim emas:

```
UDP ni tanlashning asosiy sababi — tezlik.
Lekin chat da:
  Inson xabar yozadi → Enter bosadi → xabar ketadi
  Bu jarayon 0.5-2 soniya oladi.

TCP kechiktirishi: 1-50 millisekund
Inson sezishi:     > 200 millisekund

→ TCP kechiktirishi inson seza olmaydi!
→ UDP ning tezlik afzalligi chat uchun ahamiyatsiz
```

Sabab 5 — Xavfsizlik va to'liqlik:

```
Chat xabarlari ba'zan uzun bo'ladi:
  → Fayl yuborish
  → Rasm yuborish
  → Uzun matn

UDP da katta ma'lumotlar:
  → Fragmentatsiya muammosi
  → Qismlar tartibsiz kelishi
  → Qo'lda yig'ish kerak (murakkab)

TCP da:
  → Qanchalik katta bo'lsa ham to'liq keladi
  → Biz hech narsa qilmasak ham ✅
```

---

8. Xulosa Jadvali

Xususiyat	TCP	UDP
Ulanish	✅ O'rnatiladi	❌ Yo'q
Kafolat	✅ Bor	❌ Yo'q
Tartib	✅ Saqlanadi	❌ Saqlanmaydi
Tezlik	⚠️ Sekinroq	✅ Tezroq
Overhead	⚠️ Katta (20b)	✅ Kichik (8b)
Broadcast	❌	✅
Chat	✅ Ideal	❌ Noto'g'ri
Video call	⚠️	✅ Ideal
Fayl uzatish	✅ Ideal	❌
Online o'yin	⚠️	✅ Ideal

```
Chat uchun TCP — 5 asosiy sabab:
  1. Xabar yo'qolmasligi shart
  2. Tartib saqlanishi shart
  3. Ulanish holati aniqlanishi kerak
  4. Tezlik farqi inson uchun sezilmas
  5. Katta ma'lumotlar (fayl, rasm) to'liq kelishi kerak
```

> 💡 **Eslab qol:** Protokol tanlashda asosiy savol — "Bitta paket yo'qolsa nima bo'ladi?" Chat da javob: "Xabar yo'qoladi — bu qabul qilinmas!" → TCP. Video da javob: "Bir kadr o'tkazib yuboriladi — ko'rinmaydi" → UDP.

🔴 **SAVOL 14:**

KLIENT-SERVER DASTURIDA MA'LUMOTNI `ENCODE/DECODE` QILISH ZARURIYATI. `BYTES` VA `STRING` O'RTASIDAGI FARQ, `UTF-8` KODLASH VA TARMOQ ORQALI MA'LUMOT UZATISHDA AHAMIYATI.

---

1. Nima uchun Encode/Decode Kerak?

Tarmoq — bu fizik simlar, optik tolalar, radio to'lqinlar. Ular faqat **elektr signallari**, ya'ni **0 va 1** larni uzatadi. Matn esa inson uchun — kompyuter uchun emas.

```
Inson ko'radi:   "Salom!"
Tarmoq ko'radi:  01010011 01100001 01101100 01101111 01101101 00100001

Tarmoq faqat baytlar (bytes) uzatadi — hech qachon matn emas!
```

Shuning uchun:

```
Yuborish:  "Salom!" (str) → encode() → b"Salom!" (bytes) → tarmoqqa
Qabul:     tarmoqdan → b"Salom!" (bytes) → decode() → "Salom!" (str)
```

---

2. String va Bytes — Nazariy Farq

String (str) — Matn:

```
matn = "Salom!"
type(matn)      # <class 'str'>
len(matn)       # 6 — belgilar soni

# str — abstraktsiyadir. Kompyuter xotirasida qanday
# saqlanishi kodlashga bog'liq (UTF-8, UTF-16...)
# str → faqat Python ichida ishlatiladi
```

Bytes — Baytlar:

```
bayt = b"Salom!"
type(bayt)      # <class 'bytes'>
len(bayt)       # 6 — baytlar soni (bu holatda teng)

# bytes — haqiqiy xotira ko'rinishi
# Har element 0-255 oralig'idagi son
print(list(b"Salom!"))
# [83, 97, 108, 111, 109, 33]
# bytes → tarmoq, fayl, disk uchun ishlatiladi
```

Muhim farq — Unicode belgilar:

```
# Lotin harflari — 1 belgi = 1 bayt
matn = "Hello"
print(len(matn))                    # 5
print(len(matn.encode("utf-8")))    # 5 — teng!

# O'zbekcha/Ruscha — 1 belgi = 2 bayt (UTF-8 da)
matn = "Салом"
print(len(matn))                    # 5 — belgilar
print(len(matn.encode("utf-8")))    # 10 — baytlar!
```

```python
# Arabcha, Xitoycha — 1 belgi = 3 bayt
matn = "مرحبا"
print(len(matn))                    # 5 — belgilar
print(len(matn.encode("utf-8")))    # 10 — baytlar
```

Bu farqni tushunmaslik → eng ko'p uchraydigan xato!

---

3. Kodlash Nima? — Nazariy Asos

Kodlash — bu har bir belgiga raqam berish qoidasi. Tarix davomida ko'plab kodlash tizimlari yaratilgan:

ASCII — Birinchi standart (1963):

```
Faqat 128 ta belgi: A-Z, a-z, 0-9, tinish belgilari
Har belgi = 1 bayt

'A' = 65
'B' = 66
'a' = 97
'0' = 48

Muammo: O'zbek, Arab, Xitoy harflari yo'q!
```

ISO-8859 — Mintaqaviy yechim:

```
256 ta belgi — 128 ta ASCII + 128 ta mintaqaviy
ISO-8859-1 → G'arbiy Yevropa
ISO-8859-5 → Kirill (Rus, O'zbek)

Muammo: Har mamlakat o'z kodlashi → bir fayl
         boshqa mamlakatda buzilgan ko'rinadi (krakozyabry!)
```

Unicode — Universal yechim:

```
Dunyodagi BARCHA belgilar — 143,000+ ta
Har belgiga noyob raqam (code point) beriladi:

'A'  → U+0041
'а'  → U+0430  (Kirill a)
'ا'  → U+0627  (Arab alif)
'中' → U+4E2D  (Xitoy)
'😀' → U+1F600 (Emoji!)

Unicode — raqamlar tizimi, kodlash emas!
Uni baytga aylantirish uchun UTF-8 kerak.
```

---

4. UTF-8 — Nima uchun Universal Standart?

UTF-8 — Unicode belgilarini baytlarga aylantirish usuli. Uning dahosi — **o'zgaruvchan uzunlik:**

```
Belgi turi           | Bayt soni | Misol
─────────────────────┼───────────┼──────────────
ASCII (0-127)        | 1 bayt    | 'A' → 01000001
Lotin kengaytmasi    | 2 bayt    | 'é' → 11000011 10101001
Kirill, Arab, ...    | 2-3 bayt  | 'a' → 11010000 10110000
Xitoy, Yapon, ...    | 3 bayt    | '中'→ 11100100 10111000 10101101
Emoji, qadim belgi   | 4 bayt    | '😀'→ 11110000 10011111 10011000 10000000
```

Nima uchun UTF-8 eng yaxshi?

- ✅ ASCII bilan to'liq moslik (eski dasturlar ishlaydi)
- ✅ O'zgaruvchan uzunlik — lotin matnlari kam joy oladi
- ✅ Internet standarti — barcha brauzer, server qo'llab-quvvatlaydi
- ✅ Barcha tillar — O'zbek, Rus, Arab, Xitoy...
- ✅ Self-synchronizing — o'rtadan o'qishni boshlasa ham belgilar aniqlanadi

```python
# UTF-8 encode/decode:
matn = "Salom, Дунё! 🌍"

bayt = matn.encode("utf-8")
print(bayt)
# b'Salom, \xd0\x94\xd1\x83\xd0\xbd\xd1\x91! \xf0\x9f\x8c\x8d'

qayta = bayt.decode("utf-8")
print(qayta)   # "Salom, Дунё! 🌍"
```

---

5. Tarmoqda Encode/Decode — Amaliy Jarayon

```
YUBORUVCHI (Klient):
┌──────────────────────────────────────┐
│  matn = "Salom!"          (str)      │
│  bayt = matn.encode("utf-8")  (bytes)│
│  sock.sendall(bayt)                  │
└──────────────────┬───────────────────┘
                   │
          [Tarmoq: 0 va 1 lar]
                   │
QABUL QILUVCHI (Server):
┌──────────────────▼───────────────────┐
│  bayt = sock.recv(1024)    (bytes)   │
│  matn = bayt.decode("utf-8")  (str)  │
│  print(matn)  →  "Salom!"            │
└──────────────────────────────────────┘
```

```python
# To'liq misol:

# KLIENT:
xabar = "Salom, Дунё! 🌍"
sock.sendall(xabar.encode("utf-8"))

# SERVER:
data = sock.recv(4096)
xabar = data.decode("utf-8")
print(xabar)   # "Salom, Дунё! 🌍"
```

6. Keng Tarqalgan Xatolar

Xato 1 — encode/decode unutilishi:

```
# ❌ XATO:
sock.send("Salom!")
# TypeError: a bytes-like object is required, not 'str'

# ✅ TO'G'RI:
sock.send("Salom!".encode("utf-8"))
```

Xato 2 — Noto'g'ri kodlash:

```
# ❌ XATO — ikki tomon boshqa kodlash:
# Klient:
sock.send("Привет".encode("cp1251"))   # Windows kodlashi

# Server:
data.decode("utf-8")
# UnicodeDecodeError! — cp1251 baytlari UTF-8 da noto'g'ri

# ✅ TO'G'RI — ikki tomon bir xil kodlash:
# Klient:
sock.send("Привет".encode("utf-8"))
# Server:
data.decode("utf-8")   # ✅
```

Xato 3 — Qisman ma'lumot:

```
# ❌ XATO — bitta recv() da hammasi kelmasligi mumkin:
data = sock.recv(1024)
matn = data.decode("utf-8")
# Agar "😀" belgisi ikki recv() ga bo'linsa:
# 1-recv: b'\xf0\x9f'    (to'liq emas)
# 2-recv: b'\x98\x80'
# 1-recv da decode() → UnicodeDecodeError!

# ✅ TO'G'RI — to'liq ma'lumot kelguncha kutish:
buffer = b""
while True:
    qism = sock.recv(4096)
    if not qism:
        break
    buffer += qism
    try:
        matn = buffer.decode("utf-8")
        buffer = b""
        # Matn bilan ishlash
    except UnicodeDecodeError:
        continue   # Hali to'liq kelmagan, davom et
```

Xato 4 — errors parametri:

```
# Noma'lum baytlar bo'lsa:
data = b"\xff\xfe Salom"   # noto'g'ri UTF-8 baytlar

# ❌ XATO:
data.decode("utf-8")
# UnicodeDecodeError!

# ✅ Variantlar:
data.decode("utf-8", errors="ignore")    # Noto'g'ri baytlarni o'tkazib yuborish
data.decode("utf-8", errors="replace")  # ??? belgisi bilan almashtirish
data.decode("utf-8", errors="strict")   # Xato chiqarish (standart)
```

---

7. Protokol Dizayni — Uzunlik Sarlavhasi

Tarmoqda katta xabarlar **bir necha parchalarga** bo'linib kelishi mumkin. Qancha bayt kutish kerakligini bilmasak — muammo.

Yechim — uzunlik sarlavhasi (length prefix):

```
import struct

# YUBORISH:
def xabar_yuborish(sock, matn: str):
    bayt = matn.encode("utf-8")
    uzunlik = len(bayt)

    # Avval 4 baytda uzunlikni yubor
    sarlavha = struct.pack(">I", uzunlik)   # ">I" = big-endian, 4 bayt int
    sock.sendall(sarlavha + bayt)

# QABUL QILISH:
def xabar_qabul(sock) -> str:
    # Avval 4 bayt sarlavhani o'qi
    sarlavha = _tolik_olish(sock, 4)
    uzunlik = struct.unpack(">I", sarlavha)[0]

    # Keyin aynan shuncha bayt o'qi
    bayt = _tolik_olish(sock, uzunlik)
    return bayt.decode("utf-8")

def _tolik_olish(sock, miqdor: int) -> bytes:
    """Aynan 'miqdor' bayt kelguncha kutadi"""
    buffer = b""
    while len(buffer) < miqdor:
        qism = sock.recv(miqdor - len(buffer))
        if not qism:
            raise ConnectionError("Ulanish uzildi")
        buffer += qism
    return buffer
```

Qanday ishlaydi:

```
Yuboriladi:
[0][0][0][6] [S][a][l][o][m][!]
 ↑ sarlavha   ↑ asosiy ma'lumot
```

```
 (4 bayt)     (6 bayt)

Qabul qilinadi:
1. 4 bayt o'qi → 6 deb bilinadi
2. 6 bayt o'qi → "Salom!" olinadi
```

---

8. JSON bilan Birgalikda

Real chat dasturlarida faqat matn emas, **strukturali ma'lumot** yuboriladi:

```python
import json

# YUBORISH — Python dict → JSON string → bytes:
xabar = {
    "tur":    "xabar",
    "kimdan": "Ali",
    "matn":   "Salom, Дунё! 🌍",
    "vaqt":   "14:30"
}

json_matn = json.dumps(xabar, ensure_ascii=False)
# ensure_ascii=False → Unicode belgilar qochmasdan yuboriladi

bayt = json_matn.encode("utf-8")
sock.sendall(bayt)

# QABUL QILISH — bytes → JSON string → Python dict:
bayt = sock.recv(4096)
json_matn = bayt.decode("utf-8")
xabar = json.loads(json_matn)

print(xabar["kimdan"])   # "Ali"
print(xabar["matn"])     # "Salom, Дунё! 🌍"
```

---

9. Xulosa

```
Tarmoq faqat baytlar (bytes) uzatadi — hech qachon matn emas!

str   → Python ichida matn bilan ishlash uchun
bytes → Tarmoq, fayl, disk uchun

Jarayon:
  Yuborish:  str → .encode("utf-8") → bytes → sock.send()
  Qabul:     sock.recv() → bytes → .decode("utf-8") → str

UTF-8 nima uchun?
  ✅ Universal — barcha tillar (O'zbek, Rus, Arab, Xitoy, Emoji)
  ✅ ASCII bilan mos — eski tizimlar ishlaydi
  ✅ Internet standarti — barcha tizimlar qo'llab-quvvatlaydi

Asosiy qoidalar:
  1. Har doim UTF-8 ishlatish
  2. Ikki tomon bir xil kodlash
```

```
3. Katta xabarlar uchun uzunlik sarlavhasi
4. UnicodeDecodeError → errors="replace" bilan himoya
```

> 💡 **Eslab qol:** `len("Привет") = 6` (belgilar), lekin `len("Привет".encode("utf-8")) = 12` (baytlar). Bu farqni tushunmaslik — tarmoq dasturlashda eng ko'p uchraydigan xatolardan biri!

🔴 **SAVOL 15:**

`PYQT5` DA RASMLARNI JOYLASHTIRISH USULLARI: `QPIXMAP`, `QLABEL` ORQALI KO'RSATISH, RESURSLARNI `.QRC` FAYLIDA BOSHQARISH. IKONKALAR VA FON RASMLARINI DASTURGA QO'SHISH AMALIYOTI.

---

1. PyQt5 da Rasm Ishlash Tizimi

PyQt5 da rasmlar bilan ishlash uchun asosiy klasslar:

```
QPixmap   → Diskdan rasm yuklash, ko'rsatish, o'lchamini o'zgartirish
QImage    → Piksel darajasida ishlash, rasm tahrirlash
QIcon     → Ikonkalar uchun (tugma, oyna, menyu)
QLabel    → Rasmni ekranda ko'rsatish uchun widget

Munosabat:
  QPixmap → QLabel ga o'rnatiladi → Ekranda ko'rinadi
  QPixmap → QIcon ga aylantiriladi → Tugma, oynada ko'rinadi
```

---

2. QPixmap — Rasm Yuklash va Ko'rsatish

QPixmap — ekranda ko'rsatish uchun optimallashtirilgan rasm klassi. Grafik karta xotirasida saqlanadi.

Diskdan yuklash:

```python
from PyQt5.QtGui import QPixmap
from PyQt5.QtWidgets import QLabel

# Yuklash
pixmap = QPixmap("images/rasm.png")

# Yuklash muvaffaqiyatli bo'ldimi?
if pixmap.isNull():
    print("Rasm topilmadi!")
else:
    print(f"O'lcham: {pixmap.width()} x {pixmap.height()}")
```

QLabel orqali ko'rsatish:

```python
label = QLabel()
label.setPixmap(pixmap)
```

```
# Rasmni labelga moslash
label.setScaledContents(True)    # label o'lchamiga moslashadi
label.setFixedSize(300, 200)     # label o'lchamini belgilash
```

O'lchamini o'zgartirish:

```
# Nisbatni saqlab o'lcham o'zgartirish
kichik = pixmap.scaled(
    300, 200,
    Qt.KeepAspectRatio,          # nisbat saqlanadi
    Qt.SmoothTransformation      # silliq o'zgartirish
)

# Nisbatsiz o'lcham o'zgartirish
kichik = pixmap.scaled(300, 200, Qt.IgnoreAspectRatio)

# Faqat kengligi bo'yicha
kichik = pixmap.scaledToWidth(300)

# Faqat balandligi bo'yicha
kichik = pixmap.scaledToHeight(200)
```

Rasm ustiga matn yozish (QPainter):

```
from PyQt5.QtGui import QPixmap, QPainter, QFont, QColor
from PyQt5.QtCore import Qt

pixmap = QPixmap("rasm.png")
painter = QPainter(pixmap)
painter.setFont(QFont("Arial", 20, QFont.Bold))
painter.setPen(QColor("white"))
painter.drawText(
    pixmap.rect(),
    Qt.AlignCenter,
    "Watermark"
)
painter.end()
```

---

3. QIcon — Ikonkalar

QIcon — tugmalar, oynalar, menyu elementlari uchun ikonka klassi.

```
from PyQt5.QtGui import QIcon
from PyQt5.QtWidgets import QPushButton, QMainWindow

# Oyna ikonkasi
oyna = QMainWindow()
oyna.setWindowIcon(QIcon("icons/app.ico"))

# Tugma ikonkasi
tugma = QPushButton()
tugma.setIcon(QIcon("icons/save.png"))
tugma.setIconSize(QSize(24, 24))    # ikonka o'lchami
```

```
# QPixmap dan QIcon:
pixmap = QPixmap("icons/logo.png")
ikon = QIcon(pixmap)

# Turli holatlar uchun ikonka:
ikon = QIcon()
ikon.addPixmap(QPixmap("icons/normal.png"), QIcon.Normal)
ikon.addPixmap(QPixmap("icons/hover.png"),  QIcon.Active)
ikon.addPixmap(QPixmap("icons/disabled.png"), QIcon.Disabled)
tugma.setIcon(ikon)
```

---

4. Fon Rasmi — Turli Usullar

Usul 1 — setStyleSheet orqali:

```
# Widget foni uchun
widget.setStyleSheet("""
    QWidget {
        background-image: url(images/fon.jpg);
        background-repeat: no-repeat;
        background-position: center;
    }
""")
```

Nazariy asos: QSS da `background-image` CSS kabi ishlaydi, lekin cheklovi bor — faqat `QWidget` va uning subklasslari uchun. `background-size` QSS da ishlamaydi — shuning uchun katta rasmlarda `paintEvent` usuli afzal.

Usul 2 — paintEvent orqali (professional usul):

```
from PyQt5.QtWidgets import QMainWindow
from PyQt5.QtGui import QPixmap, QPainter
from PyQt5.QtCore import Qt

class FonliOyna(QMainWindow):
    def __init__(self):
        super().__init__()
        self.fon = QPixmap("images/fon.jpg")

    def paintEvent(self, event):
        painter = QPainter(self)

        # Rasmni oyna o'lchamiga moslash
        scaled = self.fon.scaled(
            self.size(),
            Qt.KeepAspectRatioByExpanding,
            Qt.SmoothTransformation
        )

        # Markazga joylashtirish
        x = (self.width()  - scaled.width())  // 2
        y = (self.height() - scaled.height()) // 2
        painter.drawPixmap(x, y, scaled)
```

Nima uchun paintEvent afzal?

```
    setStyleSheet: rasm o'lchami o'zgarmaydi → katta/kichik ko'rinadi
    paintEvent:    oyna o'lchamiga dinamik moslashadi ✅
```

---

5. .qrc Fayl — Resurslarni Boshqarish

.qrc fayl — Qt Resource System ning markaziy elementi. Barcha rasmlar, ikonkalar, fayllar `.qrc` ga ro'yxatlanadi va Python kodiga **kompilyatsiya** qilinadi.

Nima uchun .qrc kerak?

```
    Oddiy usul (fayl yo'li):
      QPixmap("images/logo.png")
      → Dastur boshqa kompyuterda ishlasa → fayl topilmaydi!
      → PyInstaller bilan muammo

    .qrc usuli:
      QPixmap(":/images/logo.png")   ← :/ prefiksi
      → Rasm Python kodiga o'rnatiladi
      → Hamma joyda ishlaydi ✅
```

.qrc fayl yaratish:

```
<!-- resurslar.qrc -->
<!DOCTYPE RCC>
<RCC version="1.0">
    <qresource prefix="/images">
        <file>images/logo.png</file>
        <file>images/fon.jpg</file>
        <file>images/avatar.png</file>
    </qresource>

    <qresource prefix="/icons">
        <file>icons/app.ico</file>
        <file>icons/save.png</file>
        <file>icons/open.png</file>
        <file>icons/exit.png</file>
    </qresource>

    <qresource prefix="/styles">
        <file>styles/main.qss</file>
    </qresource>
</RCC>
```

.qrc → Python fayliga o'tkazish:

```
# pyrcc5 buyrug'i bilan:
pyrcc5 resurslar.qrc -o resurslar_rc.py

# Natija: resurslar_rc.py fayli yaratiladi
# Ichida barcha rasmlar base64 formatida saqlanadi
```

Python da ishlatish:

```
import resurslar_rc    # ← shu import yetarli!

# Endi :/ prefiksi bilan murojaat:
pixmap = QPixmap(":/images/logo.png")
ikon   = QIcon(":/icons/save.png")

# QSS da ham:
widget.setStyleSheet("""
    QWidget {
        background-image: url(:/images/fon.jpg);
    }
""")
```

---

6. Papka Tuzilmasi — Amaliy Yondashuv

```
loyiha/
├── main.py
├── resurslar.qrc
├── resurslar_rc.py     ← pyrcc5 yaratadi
├── images/
│   ├── logo.png
│   ├── fon.jpg
│   └── avatar.png
├── icons/
│   ├── app.ico
│   ├── save.png
│   ├── open.png
│   └── exit.png
└── styles/
    └── main.qss
```

---

7. To'liq Amaliy Misol

```
import sys
import resurslar_rc    # .qrc dan yaratilgan
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QPixmap, QIcon, QPainter, QFont, QColor
from PyQt5.QtCore import Qt, QSize


class RasmliDastur(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Rasmlar Namunasi")
        self.setFixedSize(600, 500)

        # Oyna ikonkasi — .qrc dan
        self.setWindowIcon(QIcon(":/icons/app.ico"))

        self.fon_pixmap = QPixmap(":/images/fon.jpg")
        self._interfeys_yaratish()
```

```python
def paintEvent(self, event):
    """Fon rasmi — oyna o'lchamiga moslashadi"""
    if not self.fon_pixmap.isNull():
        painter = QPainter(self)
        scaled = self.fon_pixmap.scaled(
            self.size(),
            Qt.KeepAspectRatioByExpanding,
            Qt.SmoothTransformation
        )
        x = (self.width()  - scaled.width())  // 2
        y = (self.height() - scaled.height()) // 2
        painter.setOpacity(0.3)    # fon shaffof
        painter.drawPixmap(x, y, scaled)

def _interfeys_yaratish(self):
    markaziy = QWidget()
    self.setCentralWidget(markaziy)
    tartib = QVBoxLayout(markaziy)
    tartib.setContentsMargins(30, 30, 30, 30)
    tartib.setSpacing(20)

    # === Logo rasm ===
    logo_label = QLabel()
    logo_pixmap = QPixmap(":/images/logo.png")

    if not logo_pixmap.isNull():
        scaled = logo_pixmap.scaled(
            150, 150,
            Qt.KeepAspectRatio,
            Qt.SmoothTransformation
        )
        logo_label.setPixmap(scaled)
    else:
        logo_label.setText("Logo topilmadi")

    logo_label.setAlignment(Qt.AlignCenter)
    tartib.addWidget(logo_label)

    # === Sarlavha ===
    sarlavha = QLabel("PyQt5 Rasmlar Namunasi")
    sarlavha.setAlignment(Qt.AlignCenter)
    sarlavha.setStyleSheet("""
        font-size: 22px;
        font-weight: bold;
        color: #2c3e50;
        padding: 10px;
    """)
    tartib.addWidget(sarlavha)

    # === Ikonkali tugmalar ===
    tugmalar = QHBoxLayout()
    tugmalar.setSpacing(15)

    for nom, ikon_yol in [
        ("Ochish",   ":/icons/open.png"),
        ("Saqlash",  ":/icons/save.png"),
```

```
            ("Chiqish",  ":/icons/exit.png"),
        ]:
            btn = QPushButton(nom)
            btn.setIcon(QIcon(ikon_yol))
            btn.setIconSize(QSize(24, 24))
            btn.setMinimumHeight(45)
            btn.setStyleSheet("""
                QPushButton {
                    background-color: #3498db;
                    color: white;
                    border-radius: 8px;
                    font-size: 14px;
                    padding: 0 20px;
                    border: none;
                }
                QPushButton:hover   { background-color: #2980b9; }
                QPushButton:pressed { background-color: #1a6fa8; }
            """)
            tugmalar.addWidget(btn)

        tartib.addLayout(tugmalar)

        # === Avatar — doira shaklida ===
        avatar_label = self._doira_rasm(":/images/avatar.png", 80)
        avatar_label.setAlignment(Qt.AlignCenter)
        tartib.addWidget(avatar_label)

    def _doira_rasm(self, yol: str, radius: int) -> QLabel:
        """Rasmni doira shaklida ko'rsatish"""
        o_lcham = radius * 2
        label = QLabel()
        label.setFixedSize(o_lcham, o_lcham)

        asl = QPixmap(yol)
        if asl.isNull():
            label.setText("?")
            label.setAlignment(Qt.AlignCenter)
            return label

        # Kvadrat qilib kesish
        kichik = asl.scaled(
            o_lcham, o_lcham,
            Qt.KeepAspectRatioByExpanding,
            Qt.SmoothTransformation
        )

        # Doira mask
        natija = QPixmap(o_lcham, o_lcham)
        natija.fill(Qt.transparent)

        painter = QPainter(natija)
        painter.setRenderHint(QPainter.Antialiasing)
        painter.setBrush(painter.background())

        from PyQt5.QtGui import QPainterPath
        yol_obj = QPainterPath()
        yol_obj.addEllipse(0, 0, o_lcham, o_lcham)
```

```
        painter.setClipPath(yol_obj)
        painter.drawPixmap(0, 0, kichik)
        painter.end()

        label.setPixmap(natija)
        return label


if __name__ == "__main__":
    app = QApplication(sys.argv)
    oyna = RasmliDastur()
    oyna.show()
    sys.exit(app.exec_())
```

---

8. Xulosa

```
Rasmlar bilan ishlash usullari:

QPixmap → Asosiy rasm klassi
  .scaled()          → O'lcham o'zgartirish
  .isNull()          → Yuklanganini tekshirish
  QPainter + QPixmap → Ustiga chizish

QLabel  → Rasmni ekranda ko'rsatish
  .setPixmap()       → Rasm o'rnatish
  .setScaledContents → Labelga moslashtirish

QIcon   → Ikonkalar
  Oyna, tugma, menyu uchun

.qrc fayl → Resurslarni boshqarish
  pyrcc5 resurslar.qrc -o resurslar_rc.py
  import resurslar_rc
  QPixmap(":/images/logo.png")

Fon rasmi:
  setStyleSheet → Oddiy holatlar
  paintEvent    → Professional, o'lchamga moslashuvchan

Afzalliklar:
  .qrc   ✅ Hamma joyda ishlaydi (PyInstaller bilan ham)
  yo'l   ❌ Boshqa kompyuterda topilmaydi
```

> 💡 **Eslab qol:** `.qrc` fayl ishlatish — professional yondashuv. Rasm yo'llari (`"images/logo.png"`) esa faqat development uchun. Tayyor dastur tarqatilganda `.qrc` + `pyrcc5` kombinatsiyasi eng ishonchli usul!